

Categorical predictors in regression analysis

Factor coding and its interpretation in linear regression

T. Florian Jaeger

2026-09-12

Reading and preparation for this class

Please read Gelman et al. “Regression and Other Stories” (ROS) Ch. 7.3. If you want to challenge yourself, go through problem sets 7.1 and 7.9. Then read and *work through* this document. We will use class to go through the important concepts and to address any questions that you have about the readings or the problem sets in this document.

In this document, you will find sections labeled “Prepare for class”. Please work through those examples and write up your answers. If there are a few questions, you cannot answer, elicit help on the slack channel. In addition, there are sections called “Discussion questions for class”. Please think about these questions, but don’t worry if you get stuck. These are some questions we can go through during class.

Optionally, you can read Schad, Vasishth, Hohenstein, & Klieg (2020). It is an exhaustive summary of different contrast coding schemes and their implications for regression analysis. It’s rather small-stepped and really walks you through all kinds of different scenarios you might encounter. But it’s also very long. In any case, I would definitely store it in your library for later reference—it’s the type of paper you might want to look up when you get to analyze your own data and find yourself wondering how you should code your variables. Another good reference is James et al. (2013, Ch 3.3 up to but *not* incl. 3.3.3).

About this document

This PDF is generated from an Quarto markdown document. If you're interested in understanding the code that generated the examples presented below, you can [clone the repo](#). Both the PDF and the R markdown that the PDF is generated from are in the `scripts/` folder. You can go through the document line by line, or make changes and 'knit' a new PDF with the press of a button. You can also submit corrections or suggestions for improvements as a pull request to the GitHub repository.

The data for this document

For this tutorial we will use Ashley Clark's data from her experiment on visual crowding effects on foveal processing. Here's a copy of her description of the data.

Background

Crowding is a visual phenomenon that has puzzled scientists for decades; an object in isolation can be perceived without problems, yet surrounding it with similar objects makes it harder to see. While crowding has been studied extensively in the visual periphery, humans normally orient objects in their center of gaze, the high-acuity region of the retina called the foveola. While the foveola is less than 1.5mm wide (or ~ 1 visual deg2), it contains more cones than rest of the retina combined. Human's ability to actively perceive the visual world relies heavily on not only the foveola itself, but also how and where the eye is positioned. The few studies that have examined crowding within foveal vision have produced contradictory results. Some reasons for these discrepancies include using relatively large stimuli, a small number of participants, abnormal stimuli presentation, and having indefinite stimulus presentation times with no eye tracking. More recent research, however, has highlighted the importance of precise eye tracking due to the eye's constant movement during even fixation. These small and constant movements of the eye, called fixational eye movements (FEMs), are beneficial for both high acuity vision, as well as daily tasks such as reading and face recognition. FEM's have never been examined in the context of crowding, and it remains unknown how individuals in previous crowding studies directed their foveola over stimuli, or even maintained fixation.

Goals

The goals of this research are to (1) investigate the effect of crowding within the foveola, and (2) examine if and how fixational eye movements influence crowding at this scale. Based on previous research, we hypothesize that crowding will be detrimental to foveal vision, as it is in peripheral vision, but on a finer scale. Further, based on the recent findings that FEMs are beneficial for high-acuity vision, we expect that the quality of fixational eye movements will modulate the strength of crowding within the foveola. Specifically, we predict that smaller, more precise FEMs will reduce crowding effects, while larger, less precise FEMs will increase them.

Methods

Studying FEMs during visual crowding within the foveola requires high-precision eye tracking and accuracy in localizing the center of gaze. Current video eye trackers do not have the required spatial precision, as the error of gaze localization is as large as the foveola itself. However, by using a custom built state-of-the-art eye tracking system with arcminute precision, we will be able to examine exactly how FEMs contribute to crowding within the foveola. Stimuli consist of a number-font designed specifically for studying crowding in the fovea, as it allows for recognition even when numbers are closer together than traditional optotypes used in other crowding studies.⁶ Two conditions will be examined, the uncrowded (where a single number is presented), and the crowded (where the same size number is presented, but with four surrounding numbers). The size of the number and spacing between the numbers in the crowded condition change throughout the experiment based on the subject's performance using an adaptive procedure. The stimuli presented will vary in size, ranging from 0.5 arcminutes to 4 arcminutes in width. To determine the number-width threshold, we use a standard psychophysics procedure measuring the width of the stimulus at which a subject performs above chance level.

Overview

```
# A tibble: 10 x 8
  Subject Condition Threshold DiffusionConstant Span Area Curvature Speed
  <fct>    <fct>      <dbl>          <dbl> <dbl> <dbl> <dbl> <dbl>
1 1      Uncrowded   1.39          11.1  3.13 137.   12.0  40.6
2 2      Uncrowded   1.19           4.89  2.13  57.2   15.3  38.0
3 3      Uncrowded   1.62          18.5  4.19 127.   12.7  37.6
4 4      Uncrowded   1.43          15.6  3.92 105.    9.82  45.7
5 5      Uncrowded   1.27           7.07  2.52  94.9   14.3  37.1
```

6	6	Uncrowded	1.80	23.5	3.79	131.	10.6	45.9
7	7	Uncrowded	1.62	10.8	3.09	113.	13.0	45.6
8	8	Uncrowded	1.51	18.5	3.89	101.	9.51	47.4
9	9	Uncrowded	1.75	14.5	3.21	132.	11.3	48.7
10	10	Uncrowded	1.67	14.2	4.27	281.	10.3	46.3

Code predictors explicitly!

If we want to include qualitative/categorical predictors (i.e., *factors*, e.g., “Condition A” vs. “Condition B”) in an LM, we need to translate these predictors into numerical variables.¹ This process is sometimes called predictor or factor *coding*. Some people use this term narrowly to only refer to the coding of categorical predictors. Others use it to refer to any changes or transformation we make to our predictors, including continuous predictors (e.g. centering or scaling). In this tutorial, we talk about both.

If you have been using analysis of variance (ANOVA), coding is a process that you might already be familiar with: in ANOVA-based result reporting, we regularly follow the initial ANOVA significance tests with so called “planned” or “post-hoc” comparisons. These comparisons assess hypotheses about the means of the different experimental conditions.

You might have heard terms like (forward/backward) Helmert coding, (forward/backward) sliding difference coding, polynomial coding, sum/deviation/ANOVA coding, or treatment/dummy coding, etc. These are all names for different ways to code factors, and they do exactly the same job in an LM as they do for the planned or unplanned post-hoc analyses after an ANOVA.²

In an LM, we can, and should, make these coding decision *prior* to fitting the model. Most statistics programs have default coding choices built-in that they apply to any LM or other regression you fit. **Be aware of these coding defaults**, as they will affect what the output of an LM means if you forgot to explicitly define the factor coding for the LM.³ Such defaults can be handy. But defaults can also wreak havoc. Consider, for example, that you have a predictor called `TestBlock` with values “pre” and “post”. By default, the level “post” would be coded as reference level and “pre” as treatment, which is probably *not* what you’d intuitively think. In short:

- don’t rely on default;
- code your factors explicitly;
- use variable names and level names that are transparent and avoid confusion

¹ In R, the `contrasts()` function and the family of `contr.*` functions are very helpful for factor coding (a process also known as contrast coding), without having the manually transform columns in your data table into numerical values. This approach means that the factors in the data table remain factors (allowing character values), avoiding the confusion that can result when factors are manually transformed into numeric predictors that are stored in the data table.

² In an ANOVA-based analysis approach, these post-hoc tests are the *only* way to obtain information about the *direction* of an effect, since the ANOVA itself only assesses the significance of a predictor

³ In R and Matlab, for example, all factors for which we have not explicitly defined coding will be treatment coded, with the reference level set to the first level of the factor (we’ll get to what all of this means). And, if you haven’t defined how the levels of the factor order, R will assume that they order alphanumerically.

Coding a binary categorical predictor

Let's start with the simplest case: coding a binary (two-level) factor. This will allow us to build intuitions both about *how* we can code variables, and *what the consequences of those codes are*. We'll stick with the example from above.

Factor coding essentially translates the categorical predictor into a numerical predictor. This is achieved through so called *contrasts*. Specifically, for a factor with k distinct levels we can use $k - 1$ contrasts to create $k - 1$ numerical predictors. These numerical predictors capture all the information of the k levels of the original factor. Each contrast is a vector with k numeric elements, describing the mapping from factor levels to numerical values.

For a binary factor, we thus have one contrast that is a vector with two values. This contrast defines one numerical predictor. E.g., by default in R, we'd get the following for the `Condition` predictor in our example data:

	Uncrowded
Crowded	0
Uncrowded	1

And, each time a regression function is called, this allows R to *implicitly* create a new column in our data that is the numerical translation of the `Condition` variable. Thus a model like `Threshold ~ 1 + Condition` is actually running the model `Threshold ~ 1 + ConditionUncrowded`, where `ConditionUncrowded` is a numerical variable that has been silently added to our *model matrix* X . Looking at the model matrix can be a powerful tool in understanding factor coding. For example, prior to specifying any factor coding, the model matrix for the above model would be:

```
knitr::kable(
  model.matrix(~ 1 + Condition, data = d),
  format = "latex",
  booktabs = TRUE
) %>%
  kableExtra::kable_styling(
    latex_options = "HOLD_position",
    font_size = 8)
```

(Intercept)	Condition	Uncrowded
1		1
1		1
1		1
1		1
1		1
1		1
1		1
1		1
1		1
1		1
1		0
1		0
1		0
1		0
1		0
1		0
1		0
1		0
1		0
1		0

Notice the implicitly created column of 1s for the intercept ‘predictor’ (i.e., x_0 is the constant 1). We also see that the first 10 rows of the data are coded as 1 for Condition and the second 10 rows as 0. That’s because the first 10 rows are from the uncrowded condition (see above). Below, I’ll show the model matrix for each of the coding examples. This should also help you translate the examples into another statistic program: if push comes to shove, you can always manually create the variables shown in the model matrix and then hand that manually created data to the model-fitting function of your program.

Treatment/dummy-coding

The default in many statistics programs is treatment-coding, like in the example we just went through. Under this coding scheme, we contrast all other condition against a baseline (or “reference”) condition. Since the default is in this case somewhat counter-intuitive—it’s more intuitive to think of visual *crowding* as the treatment—we set define our own treatment coding:

	Crowded
Crowded	1
Uncrowded	0

With this change in contrast, the model matrix also changes. Now the first 10 rows of the data are coded as 0 for the condition predictor, and the second rows are coded as 1.

```
knitr::kable(
  model.matrix(~ 1 + Condition, data = d),
  format = "latex",
  booktabs = TRUE
) %>%
  kableExtra::kable_styling(
    latex_options = "HOLD_position",
    font_size = 8)
```

(Intercept)	ConditionCrowded
1	0
1	0
1	0
1	0
1	0
1	0
1	0
1	0
1	0
1	0
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1
1	1

Now we can fit the LM and summarize that *fit*:

```
Call:
lm(formula = Threshold ~ 1 + Condition, data = d)

Residuals:
    Min       1Q   Median       3Q      Max
-0.58347 -0.19997 -0.03302  0.16157  1.06575

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    1.5252     0.1115   13.676 5.99e-11 ***
ConditionCrowded 0.5767     0.1577    3.657 0.0018 **
---

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3527 on 18 degrees of freedom
Multiple R-squared: 0.4262, Adjusted R-squared: 0.3943
F-statistic: 13.37 on 1 and 18 DF, p-value: 0.001805

In this treatment-coded model:

- the intercept estimate give us the model's prediction for the mean outcome *of the reference level* of Condition (because the intercept *always* gives us the model's prediction when all other terms of the model are 0—e.g., when all other predictors are 0). In this case, this is the mean threshold for the uncrowded condition.
- the estimate for `ConditionCrowded` tells us how much larger (or smaller) the threshold is for the treatment condition. In this case, this is the crowded condition.
- the sum of the intercept and the estimate for the coefficient for `ConditionCrowded` give us the model's prediction for the mean outcome of the treatment level.

We can convince ourselves that the above interpretation is correct by looking at the predictions of the model for each observation, and comparing them to the estimates for the intercept and the coefficient for `ConditionCrowded`. First, let's have a look at the model's predictions. It's just a vector with as many elements as there are outcomes in the data we fit with the linear regression. The first row below, shows the predictions for the uncrowded condition; the second row shows the prediction for the crowded condition:

1	2	3	4	5	6	7	8	9	10
1.525174	1.525174	1.525174	1.525174	1.525174	1.525174	1.525174	1.525174	1.525174	1.525174
11	12	13	14	15	16	17	18	19	20
2.101896	2.101896	2.101896	2.101896	2.101896	2.101896	2.101896	2.101896	2.101896	2.101896

Note that the model only predicts two different values—one for the crowded condition and one for the uncrowded condition. This becomes even more apparent when we attach the fitted/predicted values to the `data.frame`. Here I'm only showing the Subject, Condition, Threshold, and fitted values:

```
# A tibble: 20 x 4
  Subject Condition Threshold fitted
<fct>    <fct>      <dbl>   <dbl>
1 1      Uncrowded    1.39    1.53
2 2      Uncrowded    1.19    1.53
3 3      Uncrowded    1.62    1.53
4 4      Uncrowded    1.43    1.53
```


5	5	Uncrowded	1.27	1.53
6	6	Uncrowded	1.80	1.53
7	7	Uncrowded	1.62	1.53
8	8	Uncrowded	1.51	1.53
9	9	Uncrowded	1.75	1.53
10	10	Uncrowded	1.67	1.53
11	1	Crowded	2.11	2.10
12	2	Crowded	1.74	2.10
13	3	Crowded	3.17	2.10
14	4	Crowded	1.92	2.10
15	5	Crowded	1.52	2.10
16	6	Crowded	2.41	2.10
17	7	Crowded	2.05	2.10
18	8	Crowded	1.91	2.10
19	9	Crowded	2.31	2.10
20	10	Crowded	1.89	2.10

For the uncrowded condition, the fitted value is 1.525 ... the exact same as the intercept estimate. For the crowded condition, the fitted value is 2.101, which is 1.525 (the intercept estimate) + .576 (the estimate for the coefficient of `ConditionCrowded`). In other words, for all conditions and for all data points, the predicted/fitted value of the LM $\hat{y}_i = \text{intercept} + \text{slope} * \text{Condition}_i = \beta_0 + \beta_{\text{condition}} x_{\text{condition}}$, where $x_{\text{condition}}$ is 1 for the crowded condition and 0 for the uncrowded condition. Nice!

Prepare for class

1. How would you calculate the epsilon/residual for each observation from this data? (Try it)
2. We obtain the straightforward interpretation of the `ConditionCrowded` estimate only because we coded the treatment condition as 1, rather than, e.g., as 2. What would happen if we coded the treatment condition as 2 (and the reference condition still as 0)? Would our estimate for the coefficient for `ConditionCrowded` change? What about the estimate of the intercept? Why? (Much of this can be answered by trying to change the coding, but first probe your intuitions and then try it out)
3. Would the predicted/fitted values of the model change if we change the coding to 2 vs. 0? Why?

Questions for discussion in class

6. How could you calculate the R^2 value shown as part of the model output from just the predicted/fitted values and the original data? (cf. James et al., 2013, p. 70-71)

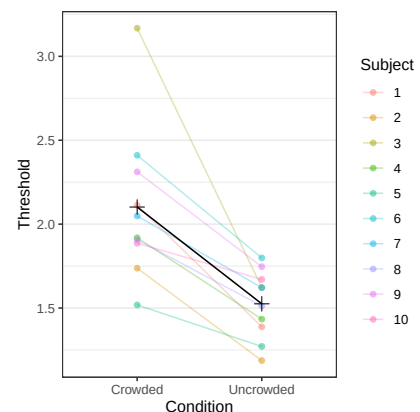


Figure 1: Size (in arcminutes) at which threshold performance (62.5% correct) was reached by crowding condition. Each color shows a separate subject. Black cross and line show the predicted outcome of a linear model fit to the data across subjects.

Effect/deviation/sum/anova-coding

While treatment-coding is the default in the regression world, it is actually rarely used in experimental psychology, brain imaging, or related fields that use factorial/balanced designs. Rather, we typically code our data in a different way, because it changes the interpretation of the coefficients in ways that researchers used to ANOVA tend to find more intuitive. This alternative is called effect/deviation/sum/anova-coding:

	Crowded.vs.Uncrowded
Crowded	0.5
Uncrowded	-0.5

With this change in contrast, the model matrix also changes:

```
knitr::kable(
  model.matrix(~ 1 + Condition, data = d),
  format = "latex",
  booktabs = TRUE
) %>%
  kableExtra::kable_styling(
    latex_options = "HOLD_position",
    font_size = 8)
```

(Intercept)	ConditionCrowded.vs.Uncrowded
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	-0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5
1	0.5

Let's refit the model with this newly coded factor:

```

Call:
lm(formula = Threshold ~ 1 + Condition, data = d)

Residuals:
    Min       1Q   Median       3Q      Max
-0.58347 -0.19997 -0.03302  0.16157  1.06575

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      1.81354    0.07886  22.997 8.54e-15 ***
ConditionCrowded.vs.Uncrowded 0.57672    0.15772   3.657  0.0018 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3527 on 18 degrees of freedom
Multiple R-squared:  0.4262,    Adjusted R-squared:  0.3943
F-statistic: 13.37 on 1 and 18 DF,  p-value: 0.001805

```

In this effect-coded model:

- the intercept estimate is now the predicted *overall mean* of the outcome (provided our data is balanced with regard to Condition).
- the estimate for the coefficient of Condition is half its distance to the mean of all conditions. It also the predicted difference between the conditions.
- the two conditions means are described as the sum of the intercept estimate $\pm \frac{1}{2}$ -times the estimate for the coefficient of Condition.

Why do researchers used to ANOVA find this more intuitive? You'll sometimes hear that it's nice that the intercept now corresponds to the mean, but the true convenience of this coding scheme will become apparent once we consider interactions below. It is under deviation-coding, that we can talk about *main effects* and *interactions* in the same sense as in an ANOVA. That's presumably also why some people call this coding scheme anova-coding. We'll get to that. But first some questions.

Prepare for class

1. What happens if we double the values we assign to each of the two effect-coded conditions, i.e., if we use 1 vs. -1 instead of .5 vs. -.5? What if we set them to -1000 vs. 1000? What changes and what doesn't? Why? (Try it)

```
contrasts(d$Condition) = cbind("Crowded vs. mean" = c(1,-1))
```

2. Are the predicted/fitted values of any of these effect-coded models different from each other? (Try it)
3. Are the predicted/fitted values of any of these effect-coded models different from the treatment-coded model presented in the previous section? Why?

Writing up results

How would we write up, for example, the effect-coded model? There are, of course, about as many difference preferences as there are researchers, and so you will find conflicting advice. But generally, it will be helpful if you are clear about all of the following:

- What model you used
- What predictors you considered⁴
- How you coded the outcome (e.g., what unit is the outcome in; without this information, readers cannot determine the size of effects or interpret them on the original scale)
- How you coded predictors (without this information, readers cannot even determine in which *direction* the effect is!)
- What steps (if any) were taken to ascertain the validity of the model (see, e.g., model evaluation in Gelman & Hill, 2007, Section 3.7). This is arguably less important if you study a well-known phenomenon that has been analyzed with this method many times, and for which you have data that is a) balanced with regard to the predictors in the model, and b) has many observations relative to the number of predictors in the model.

⁴ Including the predictors that you did decided not include in the final model (cf. debate about *researchers' degrees of freedom* as summarized in Simmons et al., 2011).

Finally, for any non-trivial model, I highly recommend the use of a summary table of the model and visualization of the *empirical* distribution of the data (potentially, while *also* showing the model's predictions). The latter helps readers not familiar with the analysis approach to understand your results.

Here is a rather detailed write-up for the model with the effect-coded condition variable. In many scenarios (e.g., if the units of the outcome variable are not necessarily informative), you might decide to provide less detail:

We analyzed the data with a linear regression, using the function `lm` from the `base` package (citation with version) of the software R (citation with version). We calculated each subject's mean thresholds for the crowded and uncrowded condition. These 20 threshold values were regressed against condition (effect-coded: $.5 = \text{crowded}$ vs. $-.5 = \text{uncrowded}$). We found a statistically significant main effect of condition ($\hat{\beta} = .576, t = 3.657, p < .01$), so that subjects reached threshold performance at a size that was about half an arcminute

larger in the crowded condition (mean = 2.102), compared to the uncrowded condition (mean = 1.525).

Combining factors and continuous predictors

Now that we know how to code factors (or at least binary factor), we can combine continuous and categorical predictors in our model. We first show a simple ‘additive’ model, in which we assume that the effects of the categorical and continuous predictors are additive. Then we consider a model that also contains an interaction, allowing the two effects to be more or less than additive.⁵

Do both the diffusion constant and the crowdedness condition affect threshold performance?

For example, let’s test the effects of both Condition and DiffusionConstant. For simplicity’s sake, we continue to use deviation coding for Condition:

$$\text{Threshold} \sim 1 + \text{Condition} + \text{DiffusionConstant}$$

What happens when we include both of these predictors in the LM?

```
Call:
lm(formula = Threshold ~ 1 + Condition + DiffusionConstant, data = d)

Residuals:
    Min       1Q   Median       3Q      Max
-0.29652 -0.13259 -0.04913  0.10420  0.44403

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      1.260450   0.097333   12.950 3.11e-
10 ***
ConditionCrowded vs. Uncrowded 0.431381   0.091055    4.738 0.00019 ***
DiffusionConstant      0.034632   0.005434    6.373 6.94e-
06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1971 on 17 degrees of freedom
Multiple R-squared:  0.8307,    Adjusted R-squared:  0.8108
F-statistic: 41.71 on 2 and 17 DF,  p-value: 2.775e-07
```

⁵ The models presented here merely serve the purpose of illustrating how we can fit, analyze, and interpret models with continuous and categorical predictors. The one degree of freedom we have in our model so far (Condition) already puts us at the maximum of the recommended degrees of freedom for 20 data points. Including additional parameters, as we do below, increases the risk of overfitting the model to the data.

Right away, we can see that the diffusion constant seems to account for a *lot* of additional variability in the model: the R^2 of the model is almost twice as large as the one we obtained when only considering condition. The output of the regression also tell us that both of the predictors have statistically significant effects on subjects' threshold performance. We can visualize the data and the model's predictions together:

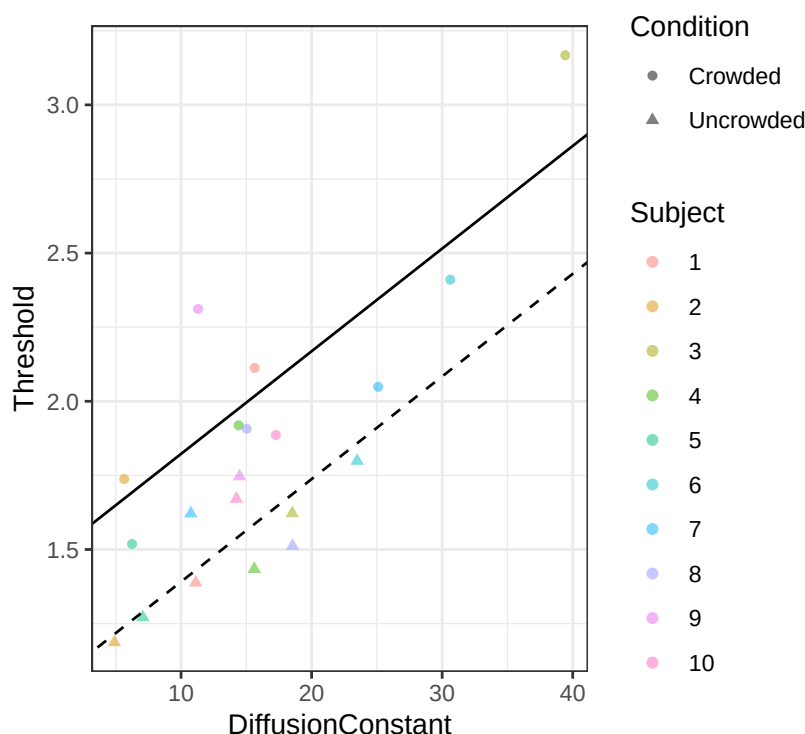


Figure 2: Size (in arcminutes) at which threshold performance (62.5% correct) was reached by diffusion constant and crowding condition. Each color shows a separate subject. Black lines show the predicted outcome of a linear model fit to the data.

Write-up

Here's a somewhat less detailed write-up for our two-predictor model, building on the example provided above:

We analyzed the data with a linear regression, using the function `lm` from the `base` package (citation with version) of the software R (citation with version). We calculated each subject's mean thresholds for the crowded and uncrowded condition. These 20 threshold values

were regressed against condition (effect-coded: $.5 = \text{crowded}$ vs. $-.5 = \text{uncrowded}$) **and the diffusion constant**. We found a statistically significant effect of condition ($\hat{\beta} = .431, t = 4.738, p < .01$), so that subjects reach threshold performance at larger sizes in the crowded condition, compared to the uncrowded condition. **We also found a statistically significant effect of the diffusion constant** ($\hat{\beta} = .035, t = 6.373, p < .01$), so that larger diffusion constants required larger sizes to achieve threshold performance.

Prepare for class

1. Notice how that the intercept estimate in this model is not the same as in the model that only contains Condition as a predictor. That also means that the intercept no longer represents the prediction for the grand mean of the outcome. Was that inevitable? Can you think of a scenario in which the inclusion of the additional predictor (`DiffusionConstant`) would *not* change the intercept estimate? (Recall that the intercept always is the model's prediction when all other terms of the model add to 0.) If you get stuck on this question, it will get resolved in the next section, but think about it before you read on.
2. The coefficient estimate for Condition has changed. Specifically, it is now somewhat smaller (.431 vs. .576). What do you make out this? Does it tell you something about the relation between the two *predictors* (`Condition` and `DiffusionConstant`)?

Discussion questions for class

4. Can we conclude from this model, and the fact that the two predictors are both significant, that the two effects are additive? Why or why not?
5. Can we conclude that `DiffusionConstant` has an effect in both crowdedness conditions? Why or why not?
6. Can we conclude that `DiffusionConstant` had an independent effect beyond condition? Why or why not?

Intermezzo: centering continuous predictors

Recall that the intercept estimate changed once we added the diffusion constant as a predictor to the LM. As we've already covered, the intercept always gives us the prediction when all other terms in the model add up to 0 (because, if $\beta_1 x_1 + \dots + \beta_k x_k = 0$ then the

LM predicts that $E(y) = \beta_0$. Partly for this reason, it is often recommended that all predictors in the model are *centered*. Since the mean of a centered predictor x_i is 0 (i.e., $E(x_i) = 0$), the average of the term $\beta_i x_i$ is also 0. So if *all* predictors in a model are centered, then $\forall i : \beta_i x_i = 0 \Rightarrow \sum_{i=1}^k \beta_i x_i = 0 \Rightarrow E(y) = \beta_0$.

For data that is balanced with regard to a factor (e.g., Condition), deviation-coding results in a centered predictor. For example, for the effect-coded model introduced above, we coded the crowded condition as .5 and the uncrowded condition as -.5. If both conditions appear equally often in the data (as they do), then the average of the implicitly created numerical predictor that results from this coding is 0.

But what about the continuous predictor in our model? `DiffusionConstant` does not have a mean of zero:

```
round(mean(d$DiffusionConstant), 5)
```

```
[1] 15.97013
```

In the combined model from the previous section, the intercept estimate thus corresponds to the threshold value that is expected when the diffusion constant is 0, which it never is because all values of the diffusion constant are positive:

```
range(d$DiffusionConstant)
```

```
[1] 4.893119 39.431606
```

Once we center diffusion constant by subtracting its mean from each of its values, the new mean of the centered diffusion constant (`DiffusionConstant_c`) is 0:

```
d %<>%
  mutate(across(where(is.numeric), list("c" = function(x) x - mean(x))))
round(mean(d$DiffusionConstant_c), 5)
```

```
[1] 0
```

When we re-fit the combined LM from the previous section with the new centered diffusion constant, the **intercept estimate now again predicts the overall mean threshold**:


```

Call:
lm(formula = Threshold ~ 1 + Condition + DiffusionConstant_c,
    data = d)

Residuals:
    Min       1Q   Median       3Q      Max
-0.29652 -0.13259 -0.04913  0.10420  0.44403

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      1.813535   0.044077   41.145 < 2e-
16 ***
ConditionCrowded vs. Uncrowded 0.431381   0.091055    4.738 0.00019 ***
DiffusionConstant_c          0.034632   0.005434    6.373 6.94e-
06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1971 on 17 degrees of freedom
Multiple R-squared:  0.8307,    Adjusted R-squared:  0.8108
F-statistic: 41.71 on 2 and 17 DF,  p-value: 2.775e-07

```

This—that the intercept estimates is the predicted grand mean of the outcome—will always be the case, no matter how many predictors we have in the model.

Note further that centering does *not* affect the slope estimates for the other effects in the model. Neither does it affect the standard error estimates of those slopes (or t statistics or p -value). Removing the mean from a predictor is a linear transformation does not affect the best-fitting slope of that predictor in predicting the outcome of an LM. Neither does centering affect the predicted/fitted values of the LM (you can compare the 20 predicted/fitted values for the combined model before and after centering `DiffusionConstant`). The same holds, of course, for the residuals, the R^2 , the adjusted R^2 and similar measures.

This will *not* always be the case. In the next section, for example, we will see that centering can change the standard error (and thus t statistic and p -value) for an interaction. More generally, centering can affect the the standard error when the removal of the mean from each variable changes the correlations between predictors (we we will return to the issue of correlations between predictors in later classes).

Additional topics we can discuss in class

Coding of continuous predictors

- scaling (dividing continuous predictors through the standard deviation). Why is this done? What are the pros and cons?
- scaling by dividing through two-times the predictors standard deviation (Gelman, 2008). Why is this done?
- non-linear transforms of continuous predictors (log-transforming, polynomials)? Why is this done? In what sense does the LM still make a linearity assumption? (see also James et al., 2013, p. 90-92)

Discussion questions for class

- Can the predictions of models differ depending on the transform of a continuous variable?
- Can the conclusions we draw about effects differ depending on the transform of a continuous variable?

Coding of factors with more than two levels

- An important distinction holds between orthogonal and non-orthogonal contrasts. Orthogonal contrasts create predictors that are orthogonal to each other, and thus have no collinearity. For balanced data sets, effect-, Helmert-, and sliding difference-coding are examples of codes that create orthogonal contrasts. Treatment-coding, on the other hand, does not create orthogonal contrasts.
- Testing hypotheses about the order of levels (see also James et al., 2013, p. 85; or this [nice summary of various coding schemes and how to make your own for R](#)). Here are some examples, using all three levels from the complete data—i.e., including the “fixation” condition:

– **treatment** coding

```
levels(d.all$Condition)
```

```
[1] "Crowded" "Fixation" "Uncrowded"
```

```
contrasts(d.all$Condition) = cbind(  
  "C" = c(1, 0, 0),  
  "U" = c(0, 0, 1))  
knitr::kable(  
  model.matrix(~ 1 + Condition, data = d.all),  
  format = "latex",  
  booktabs = TRUE
```

```
) %>%
kableExtra::kable_styling(
  latex_options = "HOLD_position",
  font_size = 8)
```

(Intercept)	ConditionC	ConditionU
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
1	1	0

- **simple slope** coding
- **effect** (“anova”) coding (and *weighted* effect coding)

```
contrasts(d.all$Condition) = cbind(
  "C.vs.F" = c(.5, -.5, 0),
  "U.vs.F" = c(0, -.5, .5))
knitr::kable(
  model.matrix(~ 1 + Condition, data = d.all),
  format = "latex",
  booktabs = TRUE
) %>%
kableExtra::kable_styling(
  latex_options = "HOLD_position",
  font_size = 8)
```

(Intercept)	ConditionC.vs.F	ConditionU.vs.F
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	-0.5	-0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.0	0.5
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0
1	0.5	0.0

- **Helmert** coding (or reverse Helmert coding) to compare weak hypotheses about the relative ordering of factor levels (each levels is compared against the mean of all preceding levels).

(Intercept)	Condition1	Condition2
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	1	-1
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	0	2
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1
1	-1	-1

- **sliding difference** coding (forward/backward) to compare stronger hypotheses about the relative order of factor levels (each level of the predictor is compared to the preceding level).

